

Technical Report: Session-aware Q&A Assistant

1. Overview

This application leverages FastAPI, LangChain (for LLM prompt chaining and retrieval), Redis (for per-session cache), FAISS (vector search), MongoDB/S3 (persistent archival), and OpenAI's GPT-4o to deliver session-based, resume-informed, multi-user conversational Q&A.

2. Key Architectural Components, with Code

2.1. Application and Middleware Setup

```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Open CORS for demo; restrict in production!
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Session Tracking Middleware

```
from uuid import uuid4

@app.middleware("http")
async def attach_session_id(request: Request, call_next):
    session_id = request.cookies.get("session_id")
    if not session_id:
        session_id = str(uuid4())
    response = await call_next(request)
    response.set_cookie("session_id", session_id)
    return response
```

Purpose: Each client is uniquely identified and tracked for chat context using a cookie.

2.2. Document Loading and VectorStore Construction

```

from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# Load, tag, and chunk the resume PDF
loader = PyPDFLoader("cv.pdf")
documents = loader.load()
for doc in documents:
    doc.metadata["source"] = "resume"

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100,
    separators=["\n\n", "\n", ".", " "]
)
docs = text_splitter.split_documents(documents)

# Embed each chunk for fast semantic search
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(docs, embeddings)

```

Purpose: Splits resume into semantically-chunked vector-embedded blocks, enabling retrieval-augmented generation.

2.3. Prompt Engineering

```

from langchain.prompts import (
    ChatPromptTemplate, SystemMessagePromptTemplate, HumanMessagePromptTemplate
)

system_prompt = (
    "You are a career assistant who gives helpful, concise answers about Naveen Prabakar "
    "based on his resume and prior questions. Any irrelevant question asked should be dismissed professionally."
)
system_msg = SystemMessagePromptTemplate.from_template(system_prompt)

human_msg = HumanMessagePromptTemplate.from_template(
    "Here is the chat history so far:\n{context}\n\nNow answer the user's latest question:\n{question}"
)

chat_prompt = ChatPromptTemplate.from_messages([system_msg, human_msg])

```

Purpose: Rigorously bounds the LLM persona to be focused, concise, and resistant to prompt drift.

2.4. Session-based Context Management in Redis

```

import redis
import json

redis_url = os.getenv("REDIS_URL")
redis_client = redis.Redis.from_url(redis_url, decode_responses=True)
SESSION_TTL_SECONDS = 600 # 10 min session

# ... inside /ask endpoint ...
session_id = http_request.cookies.get("session_id")
redis_key = f"session:{session_id}"
history_json = redis_client.lrange(redis_key, 0, -1)
history = [json.loads(msg) for msg in history_json]

# Add and cap messages
history.append({"role": "user", "content": request.query})
trimmed_history = history[-5:]

# Save back to Redis
redis_client.delete(redis_key)
for msg in trimmed_history:
    redis_client.rpush(redis_key, json.dumps(msg))
redis_client.expire(redis_key, SESSION_TTL_SECONDS)

```

Purpose: Keeps only the latest N exchanges per session, ensuring concise context window per user.

2.5. Retrieval and LLM Orchestration

```

resume_docs = vectorstore.max_marginal_relevance_search(
    request.query, k=8, lambda_mult=0.5, filter={"source": "resume"}
)
log_docs = vectorstore.similarity_search(
    request.query, k=2, filter={"source": "log"}
)
combined_docs = resume_docs + log_docs

# Compose chat context for prompt
conversational_context = ""
for msg in trimmed_history:
    role = "User" if msg["role"] == "user" else "Assistant"
    conversational_context += f"{role}: {msg['content']}\n"
injected_question = f"{conversational_context}User: {request.query}"

# Generate LLM response
response = qa_chain.run({
    "input_documents": combined_docs,
    "question": injected_question
})

```

Purpose: Augments the LLM with retrieved resume/log facts and precise, session-specific chat context.

2.6. Persistent Logging and Re-embedding

```
def log_to_s3_and_update_embeddings(question: str, answer: str):
    filename = "q&a.txt"
    local_path = f"/tmp/{filename}"

    # Download existing or create new
    try:
        s3_client.download_file(s3_bucket, filename, local_path)
    except ClientError as e:
        if e.response['Error']['Code'] == '404':
            open(local_path, "w").close()
        else:
            raise

    # Append new entry and rotate logs if >10 pairs
    # Offload old entries to MongoDB
    # ...[see previous block for details]...

    # Re-embed: Append new Q&A to vector store for next queries
    loader = TextLoader(local_path)
    new_docs = loader.load()
    new_splits = text_splitter.split_documents(new_docs)
    for doc in new_splits:
        doc.metadata["source"] = "log"
    vectorstore.add_documents(new_splits)
```

Purpose: All new Q&A is logged, persisted, and immediately retrievable thanks to periodic re-embedding.

2.7. Session Clearing Endpoint

```
@app.post("/clear_session")
def clear_session(http_request: Request):
    session_id = http_request.cookies.get("session_id")
    redis_key = f"session:{session_id}"
    redis_client.delete(redis_key)
    return {"message": "Session memory cleared."}
```

Purpose: Allows clients to securely reset all session context and chat history.

3. API Request Flow

```
@app.post("/ask")
def ask_question(request: QueryRequest, background_tasks: BackgroundTasks, http_request: Request):
    # Redis session history fetched and appended
    # Resume/log doc chunks retrieved
    # Prompt assembled
    # LLM generates response
    # Q&A appended to session; logged in background
    # New logs re-embedded for future queries
    # Response returned to client
```

4. Security and Extensibility Considerations

- **CORS is open**—lock down in prod.
- **All secrets (OpenAI, S3, Mongo, Redis)** read from environment.
- **FAISS is process-only**—consider server restarts (save/load index).
- **MongoDB archives used for cold storage and analytics**.

5. Conclusion

This backend tightly integrates Q&A logging, ephemeral session context, state-of-the-art LLM response, and cloud/hybrid vector search. Newly logged user Q&A cycles directly into the retrieval-augmented engine, enabling both accuracy and personalized knowledge graph growth—demonstrating a full-stack AI backend solution.

For further build/deploy or environment details, see the original code and ensure credentials and resource limits are configured per your production best practices.